

Minecraft 与 LittleTiles 导出 OBJ 基础技术研究文档

第三版

05/05/2025

目录

第三版前言	1
Chapter 1: Minecraft 的基础知识	3
1.1. Minecraft 中的坐标系	3
1.1.1. 世界坐标	3
1.1.2. 区块	4
1.2. Anvil——区域文件格式	4
1.2.1. Anvil 格式	4
1.2.2. 区域内的局部坐标换算	5
1.2.3. 8KiB 区	6
1.2.4. 获取区块索引与偏移量示例	6
1.2.5. 区块的读取	7
1.2.6. 获取区块 SNBT 示例	7

第三版前言

前两版的技术文档深入剖析了 **Minecraft** 与 **LittleTiles** 模组的底层技术原理，但由于配图和描述不够严谨，导致读者在理解时存在一定歧义。因此，第三版将对内容进行更为细致和准确的讲解，并配合程序代码示例详细说明。

本技术文档将与 **GitHub** 项目 **Minecraft-LittleTiles-Reader**¹（v2 分支）配合讲解，旨在提高代码的可读性。文档还将同步在 **Inception** 工作室²官网一并发布。

此外，由于第一版的 **Minecraft-LittleTiles-Reader**（主分支）在其它电脑上配置运行环境相对困难，作者正在开发第二版。该版本采用 **CMake** 进行项目依赖和构建管理，从而简化配置流程。在此，特别感谢 **GitHub** 用户 **suzakuwcx** 对该项目的贡献，尤其是在 **CMake** 部分的工作。

在查阅此文档之前需先具备基本计算机素养与知识储备，如基本的二进制及其位操作。同时读者也需至少了解一种编程语言。

*本资料 **Minecraft** 部分参考 **Minecraft Wiki**³。

¹ <https://github.com/Gaksy/Minecraft-Littletiles-Reader>

² <https://inception.work>

³ <https://zh.minecraft.wiki/>

此页留白

Chapter 1: Minecraft 的基础知识

1.1. Minecraft 中的坐标系

在 Minecraft 会有多种不同的坐标系，但通常都使用右手坐标系。我们在该文档中会涉及到**世界坐标**、**方块坐标**、**区块⁴坐标**、**子区块⁵坐标**和**区块方块坐标**等。

1.1.1. 世界坐标

世界坐标

世界坐标是由三个绝对坐标 (x, y, z) 组成的一个有序的实数三元组。

- x 表示该点在世界原点 $(0,0,0)$ 沿 x 轴方向上的有向距离；
- y 表示该点在世界原点 $(0,0,0)$ 沿 y 轴方向上的有向距离；
- z 表示该点在世界原点 $(0,0,0)$ 沿 z 轴方向上的有向距离。

这三个坐标值共同确定了该点相对于世界原点的绝对空间位置。

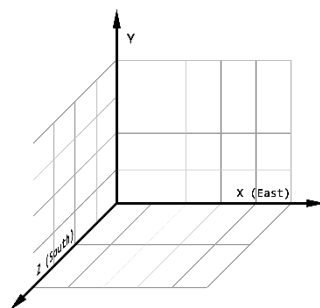


图1 右手坐标系示意图

坐标系

世界坐标基于一个由相互垂直且交于一点（即原点）的三条坐标轴形成的网格，即一个空间直角坐标系。**坐标系的单位长度为一个方块长。**

- x 轴的正方向为东，反方向为西，其坐标反映了距离原点在东（+）西（-）方向上的距离。
- z 轴的正方向为南，反方向为北，其坐标反映了距离原点在南（+）北（-）方向上的距离。
- y 轴的正方向为上，反方向为下，其坐标反映了距离原点的距离。

三条坐标轴形成了右手坐标系（拇指为 x 轴，食指为 y 轴，中指为 z 轴）。

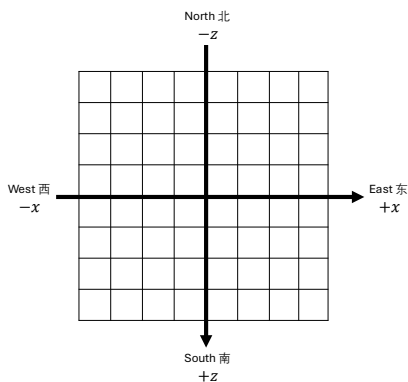


图2 坐标系轴向俯视示意图

⁴ 区块 *Chunk*

⁵ 子区块 *Sub chunk/Chunk Section*

方块坐标

一个方块的坐标实际上是这个方块的西北下角那一点的坐标，即方块内的坐标向下取整得到的整数坐标。在游戏中，一个小数坐标通常需要通过向下取整转换成整数坐标，这个整数坐标称为原坐标的方块坐标。

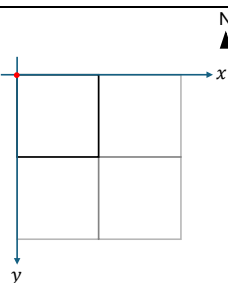


图3 方块坐标示意图

1.1.2. 区块

区块宽 16，长 16，且水平方向上与 16 整数倍的位置对齐。由于我们所探讨的版本在洞穴更新之前，故高度为 256。

区块还可以再更加细分为子区块，子区块高 16 格。

对于区块坐标，由于没有高度区分，所以由两个 32 位有符号整数⁶表示 (x, z) 两个轴向的坐标。子区块坐标使用三个 32 位有符号整数来分别表示 (x, y, z) 三个轴向的坐标。

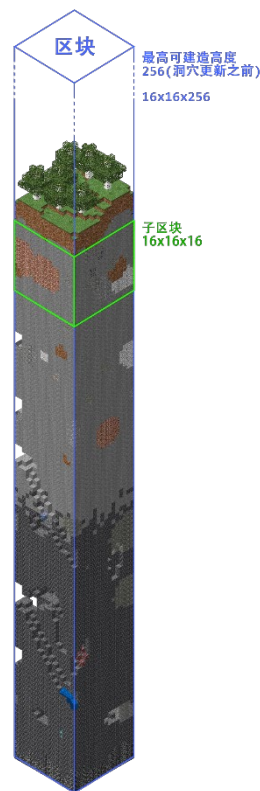


图4 区块与子区块示意图

1.2. Anvil——区域文件格式

在 1.2 版本之后，Minecraft 以 Anvil 格式存储区块数据，即 mca 文件（区域文件）。

一个区域文件存储的范围被称为区域⁷（也称为地图块）。一个区域的大小是 32×32 区块，并且区块坐标与 32 对齐。

所以区域坐标也由两个 32 位有符号整数来分别表示 (x, z) 两个轴向的坐标。

1.2.1. Anvil 格式

Anvil 格式用于指导如何存储一个地图块数据，其 mca 文件名用于指出该 mca 文件的区域坐标，如果区域的坐标为 (x, z) 则其文件命名格式为 $r.x.z.mca$ 。

区块坐标换算为区域坐标需要除以 32 并向下取整（或右移 5 位）：

```
galib::Minecraft:: ChunkCoordToRegionCoord
static_cast<std::int32_t>(std::floor(k0reChunkCoord.x / 32.0))
static_cast<std::int32_t>(std::floor(k0reChunkCoord.z / 32.0))
```

图表 1 区块坐标转区域坐标

⁶ 在 C++ 中 32 位有符号整数定义在 `cinttype` 头文件中，类型为 `std::int32_t`。

⁷ 区域 *Region*

既然区块被存储在区域文件中，虽然区块有其世界区块坐标，但为了方便在区域文件中存储，他还拥有区域内的局部坐标。

在 Anvil 格式中，前 8KiB 被称为 8KiB 区，它用于存储区块的索引数据和区块最后修改时间，剩余部分是用于存储区块已压缩数据的多个扇区。每个扇区的大小为 4096 字节，一个区块可以占用多个扇区。

因此 mca 文件大小一定为 4096 字节的倍数，Minecraft 不接受非 4096 字节的倍数的 mca 文件。

1.2.2. 区域内的局部坐标换算

局部坐标由于涉及对负数部分的处理，所以转换过程需要多几步计算。我们将涉及到负数部分的坐标做一个变换，将其视为正数处理⁸，随后我们就可以对其通过取余 32 得到对应的局部坐标。

需要注意的是，如果我们将负数轴镜像到正数轴，需要对其 -1 ，因为负数轴是从 -1 开始计数，而正数轴是从 0 开始的。

在图表 2 中， ore_x 是 x 或 z 坐标。（因为 x 与 z 都经过相同的计算） $desc_x$ 是计算所需要的中间变量，也是最终结果。

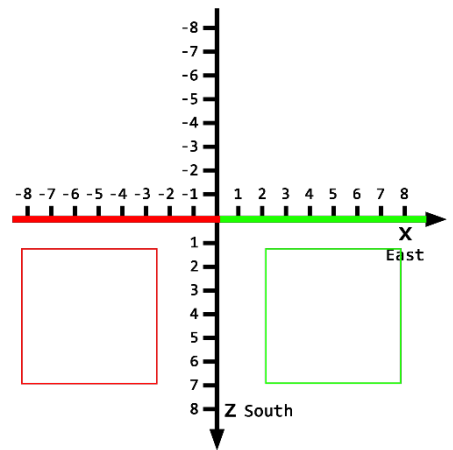


图 5 局部坐标换算示意图

$ore_x < 0$ 用以检查是否在负数轴，如果在就需要将其镜像到正数轴，反之正常取 32 的余数得到局部坐标值。

$-ore_x - 1$ 用以将值镜像到正数轴并 -1 。

随后对其得数取 32 的余数，由于我们的 $-ore_x - 1$ 操作相当于镜像到了正数轴，所以我们需要再镜像回去，所以我们对结果进行 $(32-1)-$

```
galib::minecraft::CoordSwap
NumericType desc_x;
if (ore_x < 0) {
    desc_x = -ore_x - 1;
    desc_x %= 32;
    desc_x = (32 - 1) - desc_x;
} else {
    desc_x = ore_x % 32;
}
return desc_x;
```

图表 2 局部坐标换算

$desc_x$ 计算，最终我们得出了我们所需要的局部坐标值。

但通过位运算，我们可以很简单的得出结果。我们对 x 或 z 应用按位与运算 $0x1F$ 即可得到结果，当然只适用于取 32 的余数的情况。

虽然按位与操作可以快速得出结果，但考虑到执行数量且需要计算取余 16 的情况，故该项目保留该计算方式，而非直接的按位与运算。

⁸ 这样做不会影响最终局部坐标的正确性，因为我们计算的是相对坐标，这一步相当于将其平移到了正数部分方便我们计算。

1.2.3. 8KiB 区

8KiB 区分为两个 4KiB 区，其中前者提供区块的索引相关数据，后者提供区块最后修改的时间戳（使用大端序保存纪元秒，如果未创建过则为 0）。

两个 4KiB 区都是以 4 字节为单位进行划分的，其分别存储 (x, z) 区块的偏移量和该区块的最后修改时间戳。其遍历循序应先从 x 轴开始遍历，再遍历 z 轴（ x 与 z 都从 0 开始，最大值为 31）：

$(0,0), (1,0), (2,0), \dots, (29,31), (30,31), (31,31)$

前 4KiB 区每组的前 3 个字节为偏移量，以大端序存储，单位为 4096 字节；最后一个字节为该区块所用扇区数，一个扇区为 4KiB⁹。

字节	1	2	3	4
描述	偏移量			扇区数

要计算 (x, z) 区块（世界区块坐标）在 4KiB 区中的具体字节位置，可通过如下公式的到：

$$4 \times ((x \bmod 32) + (z \bmod 32) \times 32)$$

在编程中，可能会出现负数，所以需要使用 & 运算符而不是取模运算符来进行计算：

$$4 * ((x \& 0x1F) + (z \& 0x1F) * 32)$$

1.2.4. 获取区块索引与偏移量示例

```
galib::minecraft::Anvil::getChunkConstIterator_()
// Calculating chunk offset index
const ByteIndex chunk_offset_index = 4 * kRegionChunkCoord.x + kRegionChunkCoord.z * 32;

// Calculating chunk offset
ByteIndex chunk_offset = 0;
chunk_offset = chunk_offset | static_cast<uint8_t>(kMcaData[chunk_offset_index]) << 16;
chunk_offset = chunk_offset | static_cast<uint8_t>(kMcaData[chunk_offset_index + 1]) << 8;
chunk_offset = chunk_offset | static_cast<uint8_t>(kMcaData[chunk_offset_index + 2]);
```

图表 3 计算区块索引与偏移

在项目中的“获取区块索引”的函数先计算区块在前 4KiB 区的索引获取区块的偏移量¹⁰，即 `chunk_offset_index`。

`kMcaData` 为 `mca` 文件数据，按字节读取，每一个元素一个字节。在获取区块偏移量时将其转换为 `uint8_t` 是因为读取文件时是按 `char` 类型读取的，它可能是一个有符号类型，如果对其进行位移运算会发生意想不到的效果。所以这里将其转为 `uint8_t` 之后再行位移处理。

计算区块偏移量时遵循 Anvil 文件规定使用大端序，且区块索引为三个字节。

⁹ 由于占用扇区数被保存为一个字节，所以在区域文件中区块数据最多只能保存 $255 \times 4KiB$ (1020KiB) 如果超出了这个数字，游戏会使用额外文件辅助保存这个区块。

区域文件最多保存 1024 个区块，如果所有区块都保存了 1020KiB 数据，那么可以得出区域文件最大大小为 1044488KiB(0.996GiB)。

¹⁰ 这里没有用按位与是因为事前已经计算好了区域局部坐标。

1.2.5. 区块的读取

起始扇区偏移是从文件开始计算的，即第一个区块数据的偏移为 2（扇区），剩余的区块数据偏移也至少为 2（扇区）。

如果起始扇区偏移和占用扇区数都为 0，则代表对应区块不存在或未创建。

每个区块数据的前五个字节为区块元信息，其中前四个字节存储有效数据的长度（单位为字节），最后一个字节存储压缩类型标识符：

字节	1	2	3	4	5
描述	有效数据长度				压缩类型

目前压缩类型定义了三种方案：

标识符（十进制）	方案
1	GZip（RFC1952）（未使用）
2	Zlib（RFC1950）
3（1.15.1 之前的版本）	未压缩（未使用）

当获取到有效数据长度时，可以提取其对应的压缩数据，并根据压缩类型对这段数据进行解压，即可得到 NBT 的二进制格式。

注意这里指的是原始二进制格式，而不是解析后的 SNBT 格式或解析后的树状 NBT 结构。

1.2.6. 获取区块 SNBT 示例

```
galib::minecraft::Anvil::getChunkConstIterator_()
// Get chunk length
const ByteIndex chunk_length = static_cast<uint8_t>(kMcaData[chunk_offset_index + 3]);
```

图表 4 获取扇区大小

获取区块扇区大小，以确定区块数据范围。

```
galib::minecraft::Anvil::getChunkConstIterator_()
// Get desc chunk iterator index
const ByteIndex chunk_begin_index = chunk_offset * 4096;
const ByteIndex chunk_end_index = chunk_begin_index + chunk_length * 4096;

const ByteIndex chunk_valid_begin_index = chunk_begin_index + 5;
```

图表 5 获取区块数据范围

通过区块偏移量乘以 4KiB 可获得区块起始位置（包区块元信息），然后根据扇区大小可以确定区块结束位置。

最后的区块有效起始位置就是区块起始位置增加 5 字节（区块元信息）的位置。

galib::minecraft::Anvil::getChunkConstIterator_()

```
ByteIndex chunk_valid_end_index = 0;

chunk_valid_end_index = chunk_valid_end_index | static_cast<uint8_t>(kMcadata[chunk_begin_index]) << 32;
chunk_valid_end_index = chunk_valid_end_index | static_cast<uint8_t>(kMcadata[chunk_begin_index + 1]) << 16;
chunk_valid_end_index = chunk_valid_end_index | static_cast<uint8_t>(kMcadata[chunk_begin_index + 2]) << 8;
chunk_valid_end_index = chunk_valid_end_index | static_cast<uint8_t>(kMcadata[chunk_begin_index + 3]);
ByteIndex chunk_valid_size = chunk_valid_end_index;
chunk_valid_end_index += chunk_valid_begin_index;
```

图表 6 获取区块有效数据结束位置

由于区块有效数据长度为四个字节，并以大端序存储，所以我们同样应用位操作获得有效数据长度去计算最终的区块有效数据结束位置（有效数据起始位置+有效数据长度）。